Code Layout, Readability and Reusability

Phase 5: Project Development Phase

1. Purpose of Code Organization

A readable repository lets reviewers understand how the application is built and lets contributors change one area without breaking another. trvale-book is a monorepo with a frontend folder, a backend folder, a supabase folder for the database migration and a docs folder.

2. Repository Layout

Path	Contents
frontend/src/pages	Eleven page components: Landing, Login, SignUp, Dashboard, TripWizard, ItineraryView, SavedTrips, TripEdit, Profile, Settings, NotFound
frontend/src/components	Reusable UI: Navbar, Sidebar, DemoBanner, ActivityCard, DayTimeline, BudgetSummary, ConfirmModal, Toast, SkeletonLoader
frontend/src/context	AuthContext (session, sign-in, sign-up, demo user) and TripContext (trip state)
frontend/src/services	api.js (Axios client with fallbacks), supabaseClient.js, mockData.js and its test
frontend/src/utils	formatters.js (currency, dates, duration) with tests, exportPdf.js (print helper)
backend/app/api	health.py, trips.py, export.py routers
backend/app/services	ai_service.py, budget_service.py, export_service.py
backend/app/models and core	schemas.py; config.py (settings) and security.py (token guard)
backend/tests	Five pytest modules: health, validation, AI validation, budget, trips API
supabase/migrations	001_initial_schema.sql
docs	ARCHITECTURE, DEPLOYMENT_GUIDE, SUPABASE_SETUP, E2E_TEST_CHECKLIST

3. Separation of Responsibilities

- Routers coordinate requests and responses and do not contain prompt logic.
- The AI service owns prompt construction, provider calls, parsing and the fallback generator.
- The budget service is a pure function of trip data and is independently testable.
- Schemas describe the API contract and validation rules in one place.
- The security module centralizes token verification.
- The frontend handles presentation and user actions; credentials stay on the backend.

4. Readability and Reuse

Page components are kept separate from components such as ActivityCard, DayTimeline and BudgetSummary, which are reused by the itinerary and edit pages. Formatting helpers are centralized so that currency, date and duration display is consistent. Settings are read through a typed settings class instead of scattered environment lookups.

5. Naming and Error Handling

Functions and variables are named for their purpose. The frontend wraps API calls so that a failed request falls back to demo data rather than breaking the page, and shows toast messages for user-visible errors. The backend returns specific status codes (404 for a missing trip, 403 for a trip owned by someone else, 401 for authentication problems).

6. Configuration and Secrets

Real keys are not stored in the repository. Each of the root, frontend and backend folders contains an .env.example that lists variable names with placeholder values. The .gitignore file excludes private environment files. Screenshots and logs should be checked for accidental secret exposure.

7. Improvement Opportunities

- Replace the in-memory trip dictionary with a repository layer that always reads and writes Supabase.
- Apply the declared AI_MAX_RETRIES setting in the provider calls.
- Split the large wizard page (about 650 lines) into one component per step.
- Add TypeScript types or JSDoc for API response shapes.
- Return generic error text in production instead of the raw exception message.

These are maintenance recommendations, not claims that every practice is already in place.